RAG TextLoader Knowledge Base

Generative AI and Retrieval-Augmented Generation

Generative artificial intelligence, commonly called Generative AI, refers to artificial intelligence systems that can create new content such as text, images, audio, video, and computer code. Modern generative AI systems are usually powered by large neural networks trained on very large collections of data. Large language models are a major category of generative AI models that specialize in understanding and generating natural language.

Large Language Models

A large language model, or LLM, learns statistical patterns from text during training. The model receives tokens as input and predicts likely sequences of tokens. Because the model has learned patterns from a huge amount of text, it can perform tasks such as question answering, summarization, translation, classification, code generation, and content creation.

However, an LLM does not automatically know every piece of information that a user may want. Its knowledge can be limited by its training data and knowledge cutoff. It can also produce incorrect information when it does not have enough context. Retrieval-Augmented Generation helps solve some of these problems by giving the model relevant external information at query time.

What is Retrieval-Augmented Generation?

Retrieval-Augmented Generation, or RAG, is an architecture that combines information retrieval with text generation. Instead of asking an LLM to answer a question using only its internal knowledge, a RAG system first searches a collection of documents for relevant information. The retrieved information is then provided to the language model as context.

A typical RAG pipeline has several stages. First, documents are loaded from sources such as text files, PDFs, web pages, databases, or word-processing documents. Second, the documents are divided into smaller pieces called chunks. Third, each chunk is converted into a numerical representation called an embedding. Fourth, the embeddings are stored in a vector database or vector index. When a user asks a question, the question is also converted into an embedding. The system compares the question embedding with stored document embeddings and retrieves the most relevant chunks. Finally, the retrieved chunks are passed to the LLM, which generates an answer based on the supplied context.

Document Loading

Document loading is one of the first steps in a RAG application. A document loader reads data from a particular source and converts it into a format that the rest of the pipeline can process.

In LangChain, different document loaders are available for different file types. A text file can be loaded using TextLoader. PDF files can be loaded using PDF-related loaders. Web pages can be loaded using web document loaders. CSV files, JSON files, Word documents, and other sources can also be loaded using appropriate integrations.

TextLoader

TextLoader is useful when the source data is stored in a plain text file. The loader reads the contents of the file and creates a document object. The document usually contains the actual text in its page content along with metadata describing the source.

For example, a developer can create a file named knowledge.txt and place information about a company, product, course, or technical subject inside it. TextLoader can then read this file and provide its contents to the RAG pipeline.

Why Documents Need to Be Split

A complete document can be too large to send to an LLM in a single prompt. Large documents may also contain information that is unrelated to the user's question. For this reason, RAG systems commonly split documents into smaller chunks.

Chunking improves retrieval because the vector database can search for specific sections rather than treating the entire document as one large piece. A good chunk should contain enough information to preserve meaning while remaining focused on a particular topic.

Chunk Size and Chunk Overlap

Chunk size determines approximately how much text is placed into each chunk. If chunks are too small, important context may be lost. If chunks are too large, retrieval may return too much unrelated information.

Chunk overlap means that neighboring chunks share some text. Overlap can help preserve context when an important sentence or concept occurs near the boundary between two chunks. The ideal chunk size and overlap depend on the type of documents and the application.

Embeddings

Embeddings are numerical representations of text. An embedding model converts a sentence, paragraph, or document chunk into a vector containing many numerical values. Texts with similar meanings tend to have vectors that are close to each other in embedding space.

For example, the sentences "How can I reset my password?" and "I forgot my password, what should I do?" have different words but similar meanings. A good embedding model should represent them as semantically similar vectors.

Vector Databases

A vector database stores embeddings and makes it possible to search for vectors that are similar to a query vector. Popular vector storage technologies include FAISS, Chroma, Pinecone, Weaviate, Milvus, and Qdrant.

The basic retrieval process is simple. A user's question is converted into an embedding. The vector database searches for stored vectors that are closest to the question vector. The corresponding text chunks are returned as retrieved context.

Similarity Search

Similarity search is used to identify documents that are semantically related to a query. Different mathematical methods can be used to compare vectors, including cosine similarity and Euclidean distance.

Cosine similarity measures the angle between two vectors. A value closer to one generally indicates that the vectors point in similar directions. This makes cosine similarity useful for comparing text embeddings.

RAG Question Answering

Suppose a company stores its internal documentation in a collection of text files. An employee asks, "What is the company's refund policy?" A traditional LLM may not know the company's private refund policy. A RAG application can search the company's documents, retrieve the section describing refunds, and provide that section to the LLM.

The LLM then uses the retrieved context to produce a natural-language response. In a well-designed system, the model should rely on the retrieved information instead of inventing an answer.

Hallucination

A hallucination occurs when an AI model produces information that sounds plausible but is incorrect or unsupported. RAG can reduce hallucinations by supplying relevant source information to the model. However, RAG does not guarantee that every answer will be correct.

The quality of the final response depends on multiple factors. The documents must contain accurate information. The document loader must correctly extract the content. The chunking strategy must preserve useful context. The embedding model must represent semantic meaning effectively. The retriever must find the right chunks. Finally, the LLM must correctly use the retrieved context.

Retriever

A retriever is a component that receives a user query and returns relevant documents or document chunks. A vector store can often be converted into a retriever.

A simple retriever may return the top few most similar chunks. For example, if k is set to five, the retriever may return the five chunks with the highest similarity scores. Increasing k can provide more context, but too many retrieved chunks can introduce irrelevant information and increase prompt size.

Top-K Retrieval

Top-k retrieval means selecting the k most relevant results from a collection. If k equals three, the system retrieves three chunks. If k equals ten, it retrieves ten chunks.

The correct value of k depends on the application. A question about a small product specification may require only two or three chunks. A question about a complex process may benefit from more context.

Prompt Construction

After retrieval, the application constructs a prompt containing the user's question and the retrieved context. The prompt can instruct the LLM to answer using only the supplied context.

A common instruction is to tell the model that if the answer cannot be found in the context, it should say that the information is unavailable rather than inventing an answer. This can make the application more reliable.

RAG Pipeline Example

A basic RAG pipeline can be represented as:

User Question -> Query Embedding -> Vector Search -> Relevant Chunks -> Prompt -> LLM -> Final Answer

The indexing side of the pipeline is:

Documents -> Document Loader -> Text Splitter -> Embedding Model -> Vector Store

The indexing process is usually performed before users ask questions. The retrieval and generation process happens when a user submits a query.

LangChain

LangChain is a framework and ecosystem used to build applications powered by language models. It provides abstractions for models, prompts, document loaders, text splitters, embeddings, vector stores, retrievers, and tools.

A developer can use LangChain to connect different components without implementing every integration from scratch. For example, a project can use TextLoader for a text file, a recursive character splitter for chunking, an embedding model for vector generation, a vector database for storage, and a chat model for answer generation.

Knowledge Bases

A knowledge base is a structured collection of information that an application can search. Knowledge bases can contain product documentation, company policies, educational material, technical manuals, FAQs, research papers, or customer support articles.

RAG is especially useful when the knowledge changes frequently. Instead of retraining an LLM whenever a document changes, a RAG application can update the document collection and embeddings.

Private Data

One major use case for RAG is working with private or organization-specific data. A company may have internal policies that are not present in public model training data. By connecting the model to a secure document repository, employees can ask questions about those policies.

Access control is important in private-data RAG systems. The retrieval layer should ensure that users only receive documents they are authorized to access.

Metadata

Documents can contain metadata such as source filename, document type, page number, creation date, author, department, or category. Metadata can improve filtering and retrieval.

For example, a company may have documents from multiple departments. A query about finance could be restricted to documents tagged with the finance department. This can reduce irrelevant results.

Hybrid Search

Vector search is not the only retrieval technique. Hybrid search combines semantic vector search with keyword-based search. This can be useful when exact terms, product codes, names, identifiers, or technical expressions are important.

A hybrid retrieval system may combine the results from keyword matching and semantic similarity before passing the final context to the LLM.

Reranking

A retriever may return several potentially relevant chunks, but they may not be perfectly ordered. A reranker can evaluate the query and retrieved chunks more carefully and reorder them according to relevance.

Reranking can improve retrieval quality, especially when the initial vector search returns many similar results.

Evaluation

RAG applications should be evaluated systematically. Important questions include whether the correct document was retrieved, whether the retrieved context contains the answer, and whether the generated answer is faithful to the source.

Retrieval evaluation and generation evaluation are related but different. A system can fail because it retrieved the wrong information, even if the LLM generated a perfect response from the supplied context.

Common RAG Problems

RAG systems can suffer from poor document extraction, bad chunk boundaries, low-quality embeddings, incorrect retrieval, irrelevant context, duplicated information, oversized prompts, and hallucinated answers.

Improving RAG is therefore not only about changing the LLM. Developers should inspect every stage of the pipeline. They should verify the loaded documents, examine chunks, test retrieval results, and evaluate final answers.

Production RAG

A production RAG system usually needs more than a simple notebook demonstration. It may require document ingestion pipelines, background processing, vector database management, authentication, authorization, monitoring, logging, caching, evaluation, and error handling.

Documents may also need to be re-indexed when their contents change. Some systems use document identifiers and hashes to detect changes and avoid processing unchanged documents repeatedly.

RAG and Fine-Tuning

RAG and fine-tuning solve different problems. RAG is primarily useful for giving a model access to external or changing information at inference time. Fine-tuning changes model behavior by training it further on a specialized dataset.

If the main problem is that the model needs access to frequently changing company information, RAG is often a better starting point. If the main problem is that the model needs to follow a particular style or perform a specialized behavior consistently, fine-tuning may be useful.

RAG Security

Security is an important part of enterprise RAG. Retrieved documents may contain confidential information, malicious instructions, or untrusted content. Applications should treat retrieved documents as data and carefully control what instructions the model follows.

Prompt injection is another risk. An attacker may place instructions inside a document designed to influence the model when that document is retrieved. RAG applications should use appropriate isolation, validation, access control, and model instructions.

Conclusion

Retrieval-Augmented Generation provides a practical way to connect language models with external knowledge. A simple RAG system can start with a text file, load it using TextLoader, split the content into chunks, create embeddings, store those embeddings in a vector store, retrieve relevant chunks for a question, and pass the retrieved context to an LLM.

Although the basic architecture is straightforward, high-quality RAG requires careful attention to document quality, chunking, embeddings, retrieval, prompting, security, and evaluation. Understanding each component makes it easier to debug and improve a RAG application.

Practice Questions

1. What is Generative AI?
2. What is an LLM?
3. What is Retrieval-Augmented Generation?
4. Why is RAG useful for private company data?
5. What does TextLoader do?
6. Why do documents need to be split into chunks?
7. What is chunk overlap?
8. What are embeddings?
9. What is a vector database?
10. What is similarity search?
11. What does top-k retrieval mean?
12. What is a retriever?
13. How can RAG reduce hallucinations?

14. What is hybrid search?
15. What is reranking?
16. How is RAG different from fine-tuning?
17. Why is metadata useful in RAG?
18. What are common RAG failure points?
19. Why is evaluation important in RAG?
20. What security risks can appear in RAG systems?